

Sesión L02- Clases y objetos

01

Primeros pasos con Eclipse y Maven

Eclipse es un IDE (*Integrated Development Environment*), es decir, un entorno de desarrollo, con el que podremos editar, compilar y ejecutar nuestros programas java.



Es uno de los IDEs más usados para programar en Java (y que además nos permite programar con otros lenguajes). Es un IDE muy completo, con multitud de opciones. Lógicamente, no vamos a verlas todas. Iremos introduciéndolas a medida que las vayamos necesitando.

Eclipse organiza el trabajo en **Workspaces**. Un *workspace* es un DIRECTORIO que contendrá **Proyectos**. Cada proyecto contiene el código fuente de una aplicación java que podremos compilar y ejecutar.

Cuando iniciemos eclipse por primera vez, nos propondrá el *workspace* `/home/pa/eclipse-workspace`. Usaremos dicho *workspace*.

Cada **workspace** contiene una carpeta oculta `.metadata`, que Eclipse usa para guardar la configuración de dicho espacio de trabajo, así como la lista de proyectos que contiene.

De la misma forma, cada **proyecto** en eclipse contiene un fichero oculto `.project` con información sobre la configuración relativa a dicho proyecto.

Tanto la carpeta `.metadata`, como el fichero `.project` son específicos de Eclipse (y se generan de forma automática). Por lo tanto, no nos interesa guardar en nuestro repositorio esta información. Básicamente sólo estamos interesados en guardar el código fuente (ficheros.java) y los ficheros `*.launch` que contienen los comandos que usaremos para compilar, ejecutar, borrar los ejecutables, ...)

Un elemento muy importante de este IDE son las **vistas (views)**, que se muestran en diferentes “pestañas” o “paneles”, y nos permiten mostrar nuestro proyecto desde diferentes “puntos de vista”. Las vistas están accesibles desde “**Window→Show View**”, y podemos mostrarlas u ocultarlas a voluntad, según las necesitemos o no en un momento determinado.

Algunas de las vistas que usaremos son:

Project Explorer: aquí veremos los diferentes proyectos maven del workspace, con la estructura de ficheros correspondiente.

Outline: Es una vista abstracta de nuestras clases. Es decir, muestra la información que define una clase java. Aquí veremos (cuando seleccionemos una clase java en la vista project explorer) el nombre de la clase, el paquete al que pertenece, sus atributos y sus métodos.

Console: Se abrirá automáticamente cuando compilemos/ejecutemos nuestros programas. Nos muestra todos los mensajes generados por maven durante la construcción del proyecto y también los resultados de la ejecución del mismo. La construcción de cualquier proyecto maven siempre termina con **BUILD SUCCESS** (si todo ha ido bien), o **BUILD FAILURE** (si se genera algún error durante el proceso).

Terminal: Como sugiere su nombre, es un terminal linux, que nos permitirá ejecutar comandos de forma similar al terminal (QTerminal) que lanzamos desde el escritorio de la máquina virtual..

Las vistas en Eclipse, se agrupan en **Perspectivas (Perspective)**. Una perspectiva no es más que un determinado conjunto de vistas. Siempre trabajaremos con la perspectiva **Java**.

**PROYECTOS
MAVEN**

Eclipse permite trabajar con diferentes tipos de proyectos. Uno de esos tipos son los **proyectos Maven**. Maven es una herramienta de **construcción** de proyectos (sirve para realizar de forma automática las acciones de compilación, pruebas, ejecución, ..., del código escrito en java). Maven es una herramienta muy potente y muy utilizada por los desarrolladores Java. A los proyectos que usan Maven como herramienta de construcción se les conoce como proyectos Maven.

Iremos explicando algunos conceptos de Maven y expondremos las **ventajas** de su uso. Una de esas ventajas es que define una determinada estructura (forma de organizar los ficheros del proyecto), de forma que TODOS los proyectos Maven tienen que **organizar sus ficheros** de la MISMA forma. Por lo tanto, cualquier desarrollador que conozca Maven, estará familiarizado con la estructura de los ficheros del proyecto y podrá realizar su trabajo de forma más fácil y rápida.

Un **proyecto Maven** es un directorio que contiene, **necesariamente**:

- fichero **pom.xml**: nos va a permitir construir el proyecto. El proceso de construcción recibe el nombre de **BUILD**. Podremos configurar el proceso de construcción en el pom del proyecto, e indicar qué acciones o tareas vamos a realizar y en qué orden. Ejemplos de acciones o tareas son: compilar, probar, ejecutar, desplegar ..., de forma que podremos establecer un orden entre ellas, por ejemplo: compilar/ejecutar o compilar/probar/ejecutar, o compilar/probar/desplegar, ...)
- carpetas **src/main/java**: contiene el código fuente de nuestro proyecto, formado por un conjunto de **clases** (ficheros ***.java**), organizados en **paquetes**.
- carpeta **src/main/resources**: contiene ficheros adicionales no de código, necesarios para ejecutar el proyecto (por ejemplo ficheros de imagen, o cualquier otro archivo que use nuestro programa). Estos ficheros serán copiados de forma automática por maven en la carpeta **target** durante el proceso de construcción.

Opcionalmente podemos tener las carpetas **src/test** que contendrán el código fuente de pruebas. Nosotros NO vamos a necesitar estas carpetas.

Durante el proceso de construcción del proyecto, maven genera un directorio denominado **target**, en el que se almacenarán los ficheros obtenidos durante dicho proceso, por ejemplo, los ejecutables java (ficheros ***.class**).

Comandos maven que usaremos habitualmente en las prácticas para construir el proyecto:

- **mvn clean**. Elimina el directorio **target** y todo su contenido. En cualquier caso, aunque no usemos este comando, nunca vamos a subir el directorio **target** a GitHub, ya que lo podemos generar en cualquier momento volviendo a construir el proyecto.
- **mvn compile**. Copia los ficheros del directorio **src/main/resources** en el **target**, y compila los ficheros java del directorio **src/main/java**.
- **mvn exec:java -Dexec.mainClass=<nombreClase>**. Ejecuta el método **main()** de la clase **<nombreClase>** que se encuentra en el fichero **.class** correspondiente. El método **main()** es el "punto de partida" de la ejecución de cualquier programa java. Si no hay un método **main()** NO podremos ejecutar nuestra aplicación.

**Build del
proyecto con
ECLIPSE**

Para construir el proyecto, ejecutaremos alguno de los comandos maven anteriores. SIEMPRE podemos hacerlo desde un terminal, sin necesidad de ningún IDE, aunque por comodidad lanzaremos los comandos desde Eclipse. Para lo cual tendremos que crear lo que Eclipse llama **Run Configurations**.

Dependiendo de la práctica, crearemos una o varias **Run configurations** para cada proyecto maven. Una **Run Configuration** no es más que un fichero con extensión **.launch** que contiene uno o varios comandos para construir el proyecto. Es decir, en lugar de teclear nosotros los

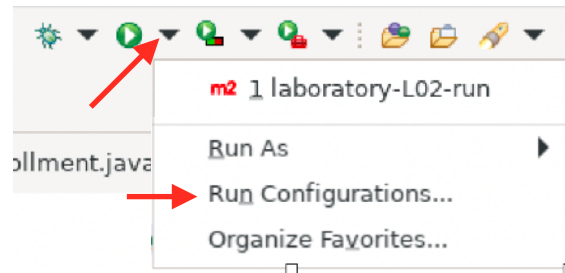
comandos maven desde un terminal, Eclipse se encargará de hacerlo por nosotros cuando pulsemos el "botón" correspondiente (asociado a alguna *Run Configuration*),

Las *Run Configurations* que creemos (ficheros con extensión `.launch`) los guardaremos en la carpeta **eclipse-launch-files**, en la raíz de nuestro proyecto maven (al mismo nivel que el `pom.xml`, y el directorio `src`).

Pasos para crear una nueva **Run Configuration** con Eclipse.

Una forma de hacerlo es: desde el menú contextual del proyecto (con botón derecho), elegimos la opción "*Run As*→*Run Configurations...*".

De forma alternativa también podemos acceder con botón izquierdo sobre el desplegable que aparece a la derecha del icono con forma de triángulo verde, y desde ahí seleccionamos "*Run Configurations...*":



A continuación, desde el panel de la izquierda, seleccionamos "*Maven Build*" y creamos una nueva *configuration* con botón derecho y la opción "*New Configuration*"

Para cada **Configuration** necesitamos indicar (desde la **pestaña Main**):

- un **nombre**
- el **directorio base** (este será el directorio raíz del proyecto maven que queramos construir). Podemos pulsar el botón **Workspace...** y seleccionar el proyecto maven.
- el comando o comandos maven que vamos a usar (campo **Goals**)
- Adicionalmente, desde la pestaña **Common**, seleccionaremos "**Shared File**" e indicaremos la ruta donde guardaremos el fichero `.launch` (que será la subcarpeta con nombre **eclipse-launch-files**, en el directorio raíz de nuestro proyecto maven).

Pasos para usar una **Run Configuration** al importar un proyecto maven.

En algunas prácticas os proporcionaremos la carpeta **eclipse-launch-files** con las *Run Configuration* ya creadas. En este caso, Eclipse cargará los ficheros correspondientes a dichas *Run Configurations* y ya tendremos todos los datos.

PERO como puede ser que vosotros trabajéis con otra carpeta de trabajo diferente, es necesario que COMPROBEMOS, **siempre que importemos un proyecto maven que os hayamos proporcionado los profesores**, si el **directorio base** y la ubicación donde se guarda el **fichero .launch** es correcta (deberá estar marcada la opción **Shared file**, de la pestaña **Common**).

Pasos para ejecutar una **Run Configuration**.

Una vez que tengamos disponible una *Run Configuration* (la hayamos creado nosotros o no, podemos ejecutarla pulsando el botón **Run**, de la ventana que contiene los datos de dicha *Run Configuration*.

Cuando se trate de una *Run Configuration* que ya hemos ejecutado como mínimo una vez, ésta nos aparecerá cuando pulsemos sobre el desplegable del botón con el triángulo verde y simplemente hacemos doble click sobre ella para ejecutarla.

En dicho desplegable siempre veremos todas las *Run configurations* que hayamos ejecutado de todos los proyectos que tengamos abiertos en el *Workspace* de Eclipse en ese momento.



Cuando importamos un proyecto maven que esté gestionado por git (y todos los proyectos de nuestro directorio de trabajo lo estarán), Eclipse lo detecta, y nos permite realizar operaciones como *git add*, *git commit*, *git push*, *git pull*... sin necesidad de usar un terminal y teclear cada vez dichos comandos.

Por comodidad, puedes usar el IDE para trabajar con GIT, pero **RECUERDA** que debes aprenderte los comandos para el examen. Mi consejo es que uses regularmente el terminal para sincronizar los cambios, de esta forma te aprenderás los comandos git más fácilmente.

Una vez que has importando el proyecto verás que Eclipse te va mostrando con diferentes iconos el estado de los ficheros de tu directorio de trabajo (por ejemplo si estos han cambiado desde el último *commit* y no están en el área *stage*, se muestran con el símbolo ">". De esta forma no necesitas usar el comando "*git status*" para ver en qué estado se encuentran dichos ficheros.

Desde el menú contextual de un archivo (o proyecto), y seleccionando la opción **Team**, podemos añadir dicho fichero (o carpeta) al stage (opción "+ **Add to index**"), hacer un *commit* (opción "**Commit ...**"), push (opción "**Push to origin**", pull (opción "**Pull**") entre otras.

Configuración de Eclipse.

Vamos a usar Eclipse porque es cómodo tener todas las herramientas necesarias para desarrollar nuestros programas en una única aplicación. Así, el IDE nos permitirá, editar, compilar y ejecutar nuestra aplicación, lo que hará que seamos más eficientes programando. Ahora bien, hay muchos IDEs, y no es buena idea depender completamente de un IDE en particular. Por lo tanto vamos a configurar ciertos parámetros en Eclipse para minimizar nuestra dependencia con dicho IDE.

Dado que estamos usando maven como herramienta de construcción de proyectos, si cambiamos de IDE, o simplemente si no disponemos de ninguno, siempre podremos ejecutar los comandos maven desde un terminal, y nuestra aplicación se compilará/ejecutará y funcionará exactamente igual que con el IDE: A continuación indicamos qué tenéis que configurar:

■ Accedemos a **Windows→Preferences→Maven→Installations**

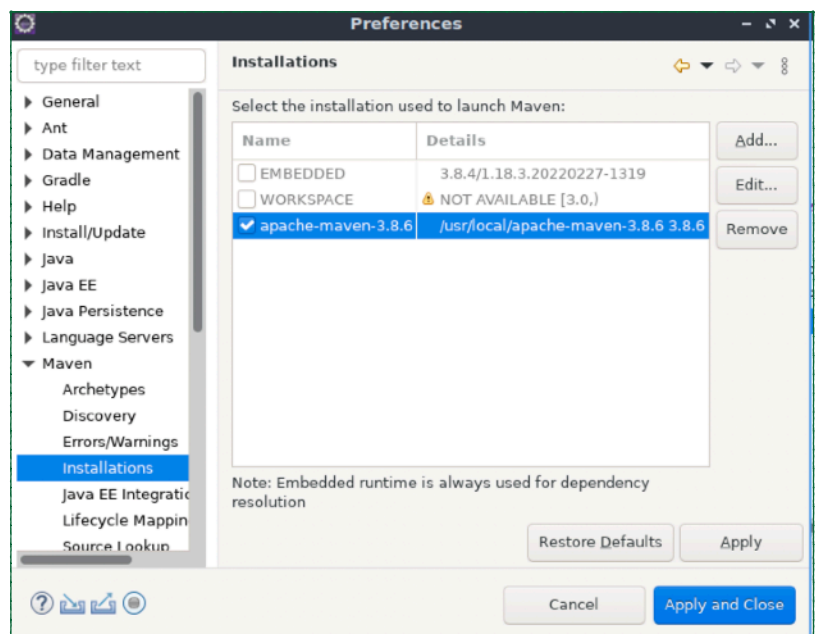
Nos aseguraremos de que tenemos marcada la línea con el valor:

/usr/local/apache-maven.3.9.3

y solamente tiene que estar esa línea seleccionada.

Eclipse incluye su propio maven. Pero queremos asegurarnos de usar la versión que nos interese, de forma que Eclipse **NO** altere en modo alguno nuestro proceso de construcción.

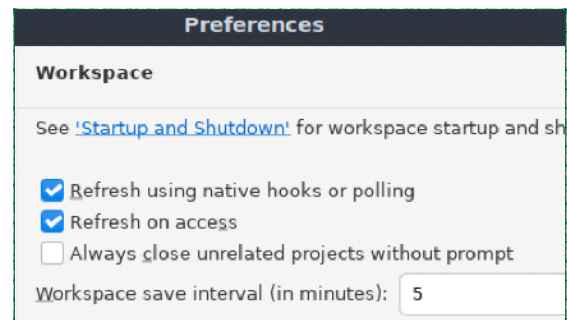
Nota: En la máquina virtual hemos instalado maven en **/usr/local/apache-maven.3.9.3**.



■ Accedemos a **Windows→Preferences→General→Workspace**

Marcaremos las opciones para "refrescar" el workspace de forma automática cuando se detecte algún cambio.

Si queremos forzar un "refresco" manual, podemos hacerlo en cualquier momento desde el menú contextual del proyecto, o de forma alternativa, pulsando la **tecla F5**.

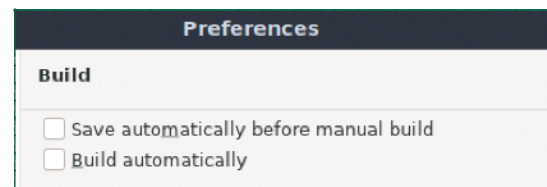


Puedes dejar marcado con 5 minutos el intervalo de tiempo en el que Eclipse guarda de forma automática el contenido de nuestro *workspace*.

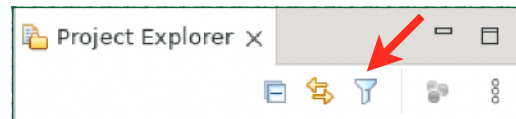
■ Accedemos a **Windows→Preferences→General→Workspace→Build**

Nos aseguraremos de que las opciones mostradas en la figura estén desmarcadas.

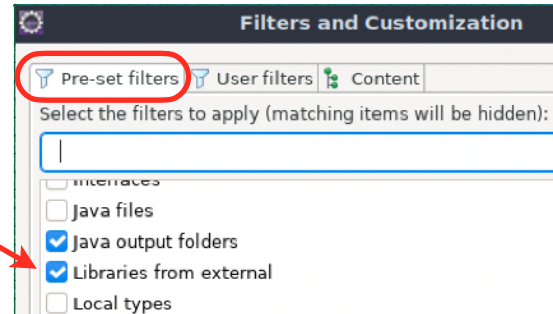
No queremos que sea Eclipse quien decida cuándo debemos guardar y/o construir nuestro proyecto.



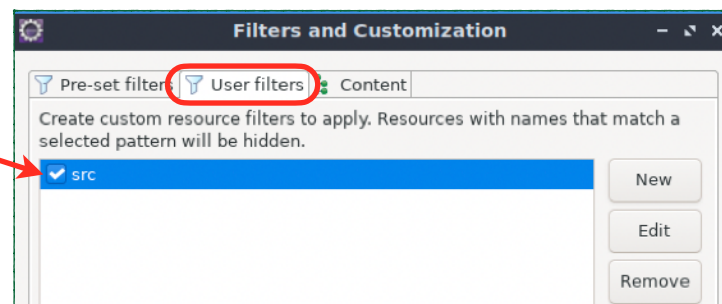
■ Desde la vista **Project Explorer**, pulsamos sobre el icono con forma de "embudo", para "filtrar" la información que se va a mostrar.



Nos aseguraremos de la opción "**Libraries from external**" esté marcada (en la pestaña "**Pre-set filters**").

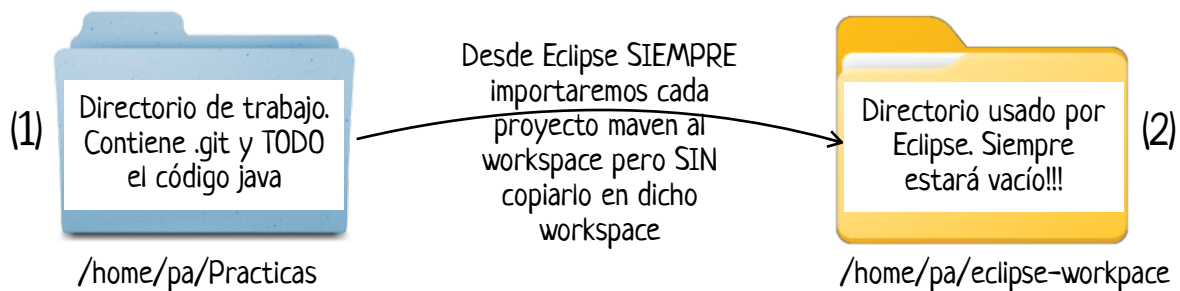


Desde la pestaña "**User Filters**" asegúrate de que tiene el filtro "**src**" y está marcado.



Forma de trabajar con ECLIPSE

Eclipse requiere trabajar siempre en un **workspace** (carpeta). Pero nosotros no queremos guardar en GitHub dicha carpeta. Por lo tanto lo que haremos será tener todo nuestro trabajo de prácticas (el directorio que contiene la carpeta oculta .git) físicamente en otro directorio DIFERENTE del workspace de Eclipse!!



TODO el código que trabajemos en prácticas estará SIEMPRE físicamente en (1). Desde Eclipse importaremos los proyectos Maven de (1) en el workspace de (2). Obviamente los cambios se guardarán en (1) porque los ficheros físicamente están ahí. Pero cuando trabajemos desde Eclipse, los "veremos" en la vista Project como si estuviesen en el workspace.

Desde (2) siempre podremos importar cualquier proyecto maven de nuestro directorio de trabajo (NUNCA lo copiaremos en el workspace). Dichos proyectos maven podremos cerrarlos, e incluso borrarlos, con la opción "Delete" del menú contextual del proyecto, pero tendremos que llevar cuidado de NO marcar NUNCA la casilla "Delete project contents on disk". Si no marcamos dicha casilla estamos "eliminando" el proyecto de (2), pero los ficheros siguen estando en (1)

Importar un proyecto maven desde Eclipse.

Siempre importaremos los proyectos maven desde Eclipse mediante la opción :

File → Import... → Maven → Existing Maven Projects.

y en el campo de texto **Root Directory** indicaremos el directorio que contiene el fichero pom.xml del proyecto maven (podemos buscarlo desde el botón **Browse**).

02

Preparamos nuestro directorio de trabajo

Para realizar esta práctica vamos a crear el directorio **L02-Clases-Objetos**, dentro de nuestro directorio de trabajo (Practicas).

```

> cd Practicas
> mkdir L02-Clases-Objetos
> cd L02-Clases-Objetos
  
```

Nota: si en algún momento haces un "clone" del repositorio remoto sin especificar un nombre de directorio, el directorio de trabajo no se llamará "Practicas". El nombre de nuestro directorio de trabajo puede ser cualquiera. Sólo tienes que tener en cuenta que tu directorio de trabajo **siempre** será el directorio que contenga la carpeta oculta **.git**.

A continuación COPIAMOS en **L02-Clases-Objetos** el contenido de la carpeta **L02-Plantillas** que os hemos proporcionado. Suponiendo dicha carpeta está en el directorio **/home/pa/Descargas**, podéis hacer la **copia** con:



```
> cp -R /home/pa/Descargas/L02-Plantillas/* .
```

Observa que hay un "espacio" y un "punto" al final del comando. Este "punto" está indicando que copiaremos todos los ficheros de `/home/pa/Descargas/L02-Plantillas` en el directorio en el que estamos (`L02-Clases-Objetos`). La opción `-R` es para indicar que queremos copiar todo lo que haya en el directorio `L02-Plantillas`, incluyendo posibles subdirectorios).

También puedes **mover** los ficheros, en lugar de copiarlos con:

```
> mv /home/pa/Descargas/L02-Plantillas/* .
```

Nuestro directorio de TRABAJO tendrá la siguiente estructura:

Directorio de Trabajo

`/home/pa/Practicas`

```
.git
.gitignore
L01-Git
└─saludo
L02-Clases-Objetos
├─dibujo-L02
├─laboratory-L02
├─ticketMachine-L02
└─zodiaco-L02
```

Cada uno de los subdirectorios de **L02-Clases-Objetos** es un proyecto Maven, y por lo tanto, obligatoriamente contiene un fichero `pom.xml`

Recuerda que en un proyecto Maven, los ficheros fuente (ficheros.java) **OBLIGATORIAMENTE** estarán en el subdirectorio `src/main/java`

Cada uno de los ficheros `.java` de cada proyecto maven contiene el código de una clase java con el mismo nombre que el fichero. Por convención, los nombres de las **clases siempre** comenzarán por una letra **mayúscula**, para así diferenciarlas de los **objetos**, que escribiremos siempre con **minúsculas**. Así, por ejemplo, la clase "Paciente" estará implementada en el fichero "Paciente.java", y su código compilado se llamará "Paciente.class". Cuando creamos un objeto de la clase *Paciente*, usaremos cualquier nombre que comience por una letra minúscula, como por ejemplo "paciente1", "pedro", o "enfermo_leve".

Recuerda también que un buen programador siempre escribe código "legible". Eso implica que las variables siempre tienen nombres representativos de lo que contienen. Por ejemplo, si tenemos dos enteros que representan un dni y un número de teléfono, podemos llamar a las variables "dni" y "telefono", en lugar de "a", "b".

03

Vamos a dibujar

Importa el proyecto maven "**dibujo-L02**" en tu *workspace* de Eclipse desde tu directorio de trabajo (`/home/pa/Practicas/L02-Clases-Objetos/dibujo-L02`). Al hacerlo, verás que, aunque en los ficheros que te proporcionamos no existe el directorio **target**, Eclipse lo crea automáticamente la primera vez que importas el proyecto.

Accede a las "Run Configuration" y comprueba si las rutas del directorio base y la ubicación de los ficheros `.launch` son correctas.

Ahora ejecuta "**dibujo-L02-clean**". Esta *Run Configuration* tiene asociado el comando `mvn clean`, que como ya hemos visto, lo que hace es BORRAR el directorio target.

Ahora vamos a compilar y ejecutar nuestra aplicación, usando la *Run Configuration* "**dibujo-L02-run**" (puedes consultar el comando maven asociado a la misma, en el campo "goals").

Verás que se imprime el mensaje por pantalla "Ejercicio pendiente de realizar".

Haciendo uso de la vista **Outline**, descubre qué métodos puedes usar con los objetos de tipo *Circle*, *Square*, *Triangle* y *Person*.

Cada vez que seleccionas una clase, automáticamente la vista *Outline* muestra la descripción abstracta de dicha clase (hablaremos sobre el concepto de abstracción más adelante).

Los nombres de los métodos de cada clase son muy descriptivos, por lo que no te hará falta entender la implementación (por ejemplo, el método `makeVisible()`, lo que hace es mostrar la figura por pantalla).

La clase **Dibujar** únicamente contiene un método `main()`, y es el método que tendrás que implementar para realizar el dibujo mostrado en el lienzo de la **Figura 1**, usando círculos, triángulos, cuadrados y una persona.

En el código del método `main()` de la clase **Dibujar**, comenta la línea en la que se imprime por pantalla el mensaje que ves al ejecutar, y crea las instrucciones necesarias, para hacer lo siguiente:

- Crea un círculo, dos cuadrados, dos triángulos y una persona (los nombres de los objetos deben comenzar por **minúscula**. Utiliza nombres como `circulo1`, `cuadrado2`, ..., de forma que "leyendo" el nombre, sea fácil averiguar a qué clase pertenece ese objeto.
- Cambia el estado de los objetos que has creado (invocando a métodos de dichos objetos) de forma que se parezcan lo máximo posible al dibujo de la **Figura 1**:



Nota: El origen de coordenadas del lienzo es la esquina superior izquierda del dibujo. Así, por ejemplo `moveHorizontal(-8)` mueve el objeto hacia la izquierda, con `moveVertical(10)` nos movemos hacia abajo

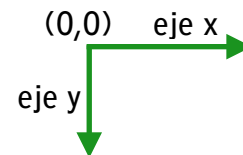


Figura 1. Objetos y estado final

Deberás usar las **Run Configurations** para eliminar el *target* y compilar y ejecutar tu código.

Cuando termines el ejercicio, sube a GitHub el trabajo realizado hasta el momento con:

```
> git add . //desde el directorio que contiene el repositorio git
> git commit -m"Terminado el ejercicio (proyecto dibujo) de L02"
> git push
```

04

Métodos: paso de parámetros y retorno de resultados

Importa el proyecto Maven **laboratory-L02**. Revisa primero las Run Configurations para asegurarte de que son correctas. Proporcionamos la clase **Enrollment**. Tendrás que comentar la única línea del su método `main()`, e implementar lo siguiente:

Nota: tendrás que usar sentencias `System.out.println` para incluir las líneas en rojo mostradas en el resultado de la ejecución de tu programa

Para imprimir un String por pantalla debes usar las sentencias:

```
System.out.println("imprime la cadena y un retorno de carro");
System.out.print("imprime la cadena sin retorno de carro");
```

Para imprimir un retorno de carro, puedes incluir `\n` en el String:

```
System.out.print("línea1 \n\nlínea2 \n\n");
```

Puedes usar tantos `\n` como necesites.



- **Crea** varios estudiantes e **imprime** sus datos por pantalla (usa el método *print()* de la clase *Student*). A continuación indicamos los datos para crear dichos estudiantes:

Nombre: "Monica Geller", ID: "A00234", créditos: 24
Nombre: "Joe Tribiani", ID: "C22044", créditos: 56
Nombre: "Chandler Bing", ID: "A12003", créditos: 6
Nombre: "Rachel Green", ID: "B66003", créditos: 12

Nota: Las cadenas de caracteres en Java son de tipo String y SIEMPRE se indican entre **comillas dobles**.

- Crea dos objetos de tipo **LabClass**, con la siguiente información:
 - ➔ una clase tendrá una capacidad para 2 estudiantes, como profesor a "Eli", con horario (atributo *timeAndDay* de la clase) "Miércoles, 15h", y se impartirá en el "Aulario 2"
 - ➔ la otra clase tendrá una capacidad para 1 estudiante, como profesor a "Jose Antonio", con horario "Miércoles, 17h", y se impartirá en el "Aulario 2".
- Usa el método **enrollStudent** definido en la clase *LabClass* para añadir los dos primeros estudiantes a la clase de "Eli" y los otros dos a la clase de "Jose Antonio".
- Imprime por pantalla los listados de cada clase, invocando al método **printList()**

Para imprimir un String por pantalla debes usar las sentencias:

```
System.out.println("imprime la cadena y un retorno de carro");  
System.out.print("imprime la cadena sin retorno de carro");
```

Cuando ejecutes el programa debes ver por pantalla el resultado que mostramos en la imagen de la derecha:

Recuerda que:

- lo que se muestra en **negro** es el resultado de invocar a alguno de los métodos de los objetos que usas en tu método *main()*, y
- lo que se muestra en **rojo** es el resultado de usar sentencias *System.out.println()*, que deberás incluir en el código del método *main()*

Students'list:

```
Monica Geller, ID: A00234, credits: 24  
Joe Tribiani, ID: C22044, credits: 56  
Chandler Bing, ID: A12003, credits: 6  
Rachel Green, ID: B66003, credits: 12
```

Enroll students (Eli):

```
Monica Geller added  
Joe Tribiani added
```

Enroll students (Jose Antonio):

```
Chandler Bing added  
The class is full, Rachel Green cannot enrol.
```

```
-----  
Lab class Martes, 15h  
Instructor: Eli Room: Aulario 2  
Number of students: 2  
Monica Geller, ID: A00234, credits: 24  
Joe Tribiani, ID: C22044, credits: 56
```

```
-----  
Lab class Martes, 17h  
Instructor: Jose Antonio Room: Aulario 2  
Number of students: 1  
Chandler Bing, ID: A12003, credits: 6
```

Sube a GitHub el trabajo realizado hasta el momento con:

```
> git add . //desde el directorio que contiene el repositorio git  
> git commit -m"Terminado el ejercicio laboratory de L02"  
> git push
```

05

Estructura de clases y sentencias condicionales

Importa el proyecto Maven **ticketMachine-L02** y revisa las *Run Configurations* para asegurarte de que son correctas. La clase **pa.TicketMachine** representa una máquina expendedora de tickets de un determinado precio. El precio de los tickets se determina cuando se crea una instancia de tipo *TicketMachine*, de forma que cada nueva instancia puede vender tickets con un precio diferente.

El usuario podrá introducir dinero en la máquina, y podrá comprar un ticket (si es que ha introducido un importe suficiente, en el método *printTicket()*), o pedir la devolución del dinero introducido, consultar dicho dinero introducido, ... Mira los comentarios añadidos a cada uno de los métodos de la implementación de la clase *TicketMachine*.

Vamos a realizar cambios en el código de la clase **TicketMachine**.

- Añade un nuevo método denominado **refundBalance()**, que devuelve el dinero que el usuario ha ido introduciendo en la máquina, y que no ha sido usado (no ha sido "cobrado" todavía). Este método ejecuta la misma acción que una máquina real, cuando pulsamos el botón que nos devuelve el dinero que hay en el cajetín. Fíjate que una vez que nos devuelve el dinero, el "cajetín" (atributo *balance*) se queda "vacío".

- Modifica el código del método **printTicket()**. En este método es donde se hace efectiva la compra (si hay dinero suficiente) y se imprime el ticket por pantalla. Si el importe no es suficiente, NO se imprimirá el ticket (ni se cobrará su importe) y se debe mostrar el mensaje "Introduzca al menos x céntimos más", siendo "x" la cantidad que falte hasta alcanzar el precio del ticket.

Si hemos introducido dinero de más, entonces mostraremos al usuario el mensaje:

"Le sobran x céntimos. Puede solicitar la devolución", en donde x es el valor del dinero que hay de más.

Tendrás que modificar el valor de los atributos **total** y **balance** para hacer efectiva la compra del ticket.

Ahora modificaremos el código del método **pa.PruebaMaquinas.main()** en el que implementaremos lo siguiente:

- Crea 3 instancias de máquinas expendedoras de tickets, con tres precios diferentes: 20, 30 y 40, respectivamente. Imprime por pantalla los mensajes de la derecha. Para mostrar el valor del ticket usa el método *getter* que corresponda..

```
Maquina 1. Precio ticket = 20
Maquina 2. Precio ticket = 30
Maquina 3. Precio ticket = 40
```

Nota: el operador + permite concatenar cadenas y también valores numéricos. Por ejemplo, supongamos que hemos declarado la siguiente variable:

```
int num = 10;
```

La expresión: ("El valor consultado es: "+num) devuelve "El valor consultado es: 10"

- Para **cada una** de las máquinas (ver la traza de ejecución de la **Figura 2**):
 - ➔ imprime un mensaje indicando el dinero que vas a introducir cada vez (el valor del dinero introducido debes obtenerlo usando los *getters* correspondientes). Mira las cantidades en la **Figura 2**. A continuación Introduce dinero usando el método **insertMoney()**.
 - ➔ Muestra el valor del dinero introducido hasta ese momento (valor de balance) e **imprime el ticket** (mira la **Figura 2**).
- A continuación en la **máquina 2** introduce 20 céntimos y pide el ticket. Fíjate que esta máquina tiene que imprimir el ticket a pesar de que hemos introducido 20 céntimos y el precio del ticket es 30.

- Muestra por pantalla el total recaudado por **cada máquina**.
- Finalmente introduce de nuevo 80 céntimos en la **máquina 2** y solicita la devolución. Mostraremos de nuevo en pantalla el total recaudado por cada máquina. Fíjate que debemos obtener el mismo resultado que antes, ya que no hemos comprado más tickets.

A continuación, en la **Figura 2** mostramos la salida completa por pantalla resultante de ejecutar el programa. En rojo se muestran las sentencias `System.out.println()` que debes usar en el método `main()` de la clase **pa.PruebaMaquinas**:

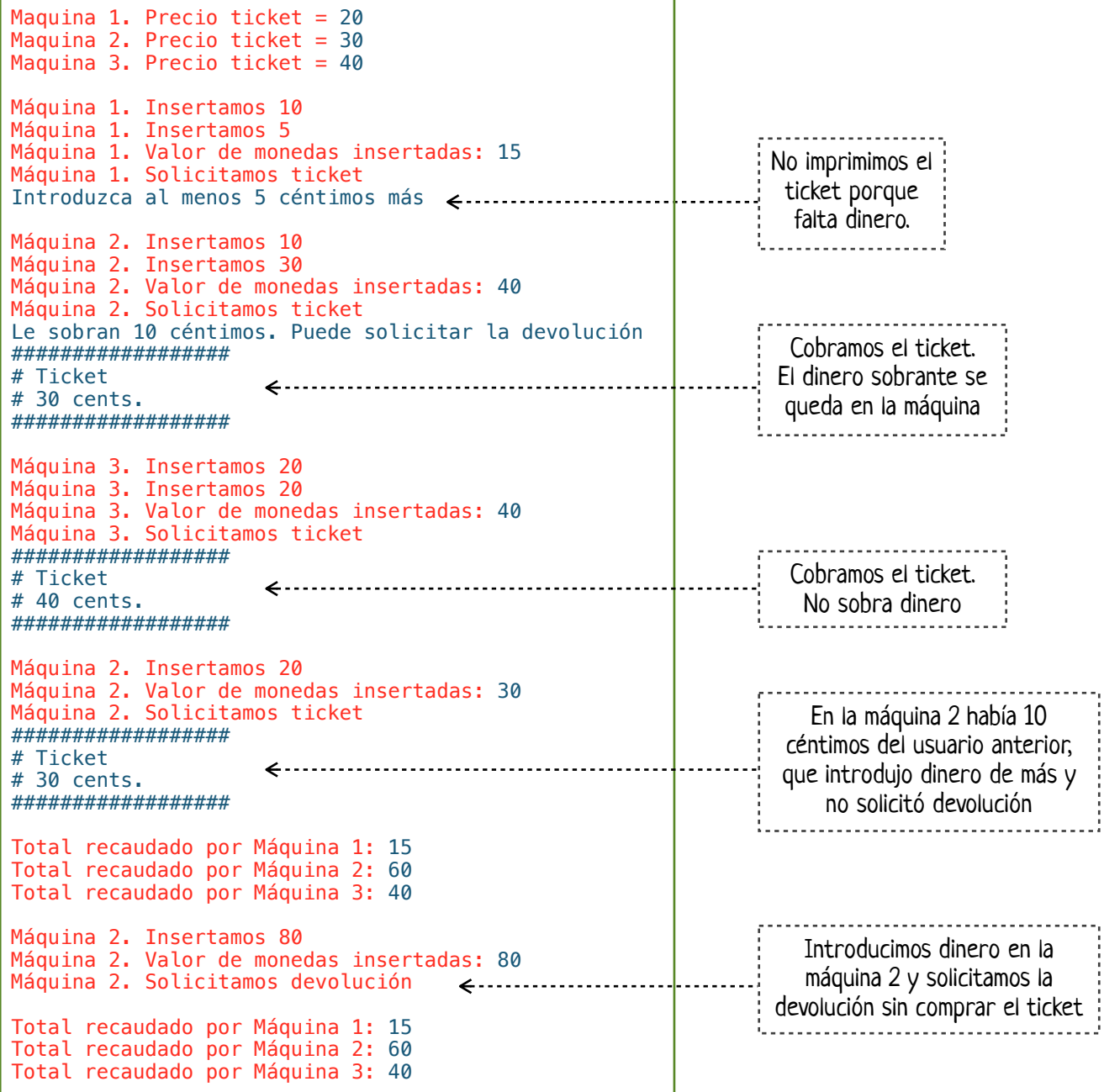


Figura 2. Resultado de la ejecución del programa

Sube a GitHub el trabajo realizado hasta el momento:

```

> git add . //desde el directorio que contiene el repositorio git
> git commit -m"Terminado el ejercicio (proyecto ticketMachine) de L02"
> git push
  
```



06

Sentencia *switch*

Importa el proyecto Maven **zodiaco-L02** y revisa las *Run Configurations* para asegurarte de que son correctas. Vamos a trabajar con la clase *Zodiaco*. Cada nueva instancia de esta clase (objeto), se inicializa con los valores de día (1..31) y mes (1..12) que se pasan como parámetros del constructor. Inicialmente la propiedad “signo” tiene el valor “Desconocido”.

La clase *Prueba*, contiene el método estático **main()**, que usaremos para probar nuestro código. Se pide:

- Implementa el código del método **fecha_valida()**. Es un método **privado** (que solamente es accesible desde el código de la clase) que devuelve cierto o falso en función de que la combinación de valores de día y mes sean válidos o no. Supondremos que el mes de febrero tiene 28 días (no se trata de un año bisiesto). Debes usar la sentencia *switch*.
- Implementa el código del método **obtener_signo()**. Es un método **público** (accesible desde fuera de la clase), que devuelve el nombre del signo asociado al día y mes de cada objeto de tipo *Zodiaco*. Puedes consultar la **tabla B** para ver la lista de signos. Para devolver el signo, previamente tendrás que validar la fecha. Debes usar la sentencia *switch*. Y sólo debes usar un “return” al final del método.
- Implementa el método **Prueba.main()** para probar el código anterior. Debes crear varias instancias de tipo *Zodiaco* (a partir de los datos de la **Tabla A**), y mostrar el signo correspondiente (o la indicación de que la fecha no es válida). La **tabla B** muestra las fechas válidas asociadas a cada símbolo del zodiaco.

Día	Mes	Resultado
23	9	Virgo
9	40	Fecha inválida
-3	8	Fecha inválida
23	11	Sagitario
21	1	Acuario
23	8	Leo

Tabla A. Valores de prueba

SIGNO	FECHAS
ARIES	21 de marzo – 20 de abril
TAURO	21 de abril – 21 de mayo
GÉMINIS	22 de mayo – 21 de junio
CÁNCER	22 de junio – 23 de julio
LEO	24 de julio – 23 de agosto
VIRGO	24 de agosto – 23 de septiembre
LIBRA	24 de septiembre – 23 de octubre
ESCORPIO	24 de octubre – 22 de noviembre
SAGITARIO	23 de noviembre – 21 de diciembre
CAPRICORNIO	22 de diciembre – 20 de enero
ACUARIO	21 de enero – 19 de febrero
PISCIS	20 de febrero – 20 de marzo

Tabla B. Signos del zodiaco con sus fechas

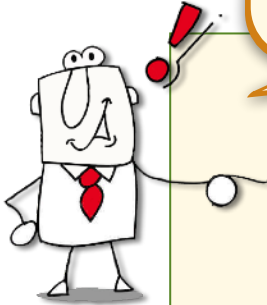
A la derecha mostramos un ejemplo de ejecución de *Prueba.main()* con el resultado que debe aparecer en pantalla cuando usamos los datos de prueba de la tabla A:

```
23,9 --> Virgo
9,40 --> Fecha inválida
-3,8 --> Fecha inválida
23,11 --> Sagitario
21,1 --> Acuario
23,8 --> Leo
```

Sube a GitHub el trabajo realizado hasta el momento:

```
> git add . //desde el directorio que contiene el repositorio git  
> git commit -m"Terminado el ejercicio (proyecto zodiaco) de L02"  
> git push
```

¿Qué **conceptos** y **cuestiones** me deben quedar CLAROS después de hacer la práctica?



- Las diferentes vistas de Eclipse: Project Explorer, Outline, Editor, Console
- La estructura de ficheros de un proyecto Maven
- La forma de ejecutar un proyecto Maven desde Eclipse
- ¿Qué contiene el directorio target y por qué no es necesario guardarlo en el repositorio git?
- ¿Qué contiene cada fichero .java y por qué no puede ejecutarse dicho fichero?
- La diferencia entre una clase y un objeto
- Las diferentes "partes" de una clase: atributos (propiedades) y métodos.
- Las diferencias entre constructores y métodos accessor y mutator.
- Cómo se crean nuevos objetos en Java
- Un objeto siempre se almacena por referencia, mientras que los tipos primitivos se almacenan por valor
- Qué es el estado de un objeto
- Cómo se inicializa y cambia el estado de los objetos en java
- Cómo podemos interactuar con los diferentes objetos
- Para qué sirve el método main()
- Cómo se pasan los parámetros en Java y para qué sirven
- Cuál es el flujo de control de una sentencia condicional (if y switch)
- Los atributos de una clase los definiremos como privados. La palabra reservada "private" indica que dicho elemento no puede ser accedido desde fuera de la clase (o del lugar en el que haya sido definido dicho elemento)
- Los métodos de una clase tienen acceso a todos los atributos de dicha clase
- Los métodos de una clase pueden invocarse directamente desde el código de dicha clase
- Un método siempre pertenece a una clase. Por lo tanto, para referirnos a dicho método siempre lo haremos indicando la clase a la que pertenece, p.ej. ClaseX.metodo1() indica que el método "metdodo1" pertenece a la clase "ClaseX"